VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



CO5085 ASSIGNMENT A1 REPORT

CNN VS TRANSFORMER ON CIFAR-100

Group: Traiblazer

Instructors: Dr. Le Thanh Sach
Dr. Nguyen Quoc Minh

Student: Dinh Hoang Duy Khanh 2670306

Ho Chi Minh City, November 2026

Member List and Workload

No.	Name	Student ID	Main responsibilities	Contribution
1	Dinh Hoang Duy Khanh	2670306	Select the dataset, implement the eight A1 experiments, run training, create figures, update GitHub Pages, and write this report.	100%

Contents

Member List and Workload **2**

Introduction and Assignment Goal **4**
A1 requirement checklist 5

1 CIFAR-100 Dataset and Protocol **6**
1.1 Why this dataset is suitable 6
1.2 Split, loader, and augmentation 6
1.3 Data representation and evaluation metrics 7

2 ResNet-18 and Swin-T Theory **8**
2.1 ResNet-18: the CNN model 8
2.2 Swin-T: the Transformer model 9
2.3 Why pretrained weights matter 11

3 Transfer Learning and Freeze Modes **12**
3.1 The four configurations 12
3.2 What freezing changes 12
3.3 Experiment Algorithm 12

4 Measured Results and Discussion **14**
4.1 Execution environment 14
4.2 Freeze comparison and training cost 15
4.3 Official test evaluation after validation selection 16

5 Error Analysis, Limitations, and Reproduction **17**
5.1 Confusion matrix and wrong predictions 17
5.2 Limitations 18
5.3 How to reproduce 18

6 Conclusion **19**

References **20**

Introduction and Assignment Goal

Assignment A1 asks for a fair comparison between a CNN and a Transformer on a public image dataset that is larger and harder than the E1 dataset. The comparison must include random initialization and ImageNet pretraining. For each pretrained model, it must include three choices: train the classifier head only, train part of the backbone, and train all layers. It must also report accuracy or F1, training cost, and error analysis.

This project uses CIFAR-100. The code compares ResNet-18, a CNN, with Swin-T, a hierarchical vision Transformer. There are eight experiments: four for each family. Every experiment uses the same train-validation split and the same three-epoch budget. Validation accuracy selects one configuration. The untouched official test set is used only after that selection. This rule prevents a test result from deciding which model looks best.

Main measured result. The validation winner is ImageNet-pretrained Swin-T with full fine-tuning. It obtains validation accuracy 0.8281. One final run on the official CIFAR-100 test set gives accuracy 0.8173 and macro-F1 0.8162. These numbers are measured by `A1/run_a1.py`; they are not estimated values.

A1 requirement checklist

Assignment requirement	Evidence in this project
Large public image dataset	CIFAR-100 has 50,000 official training images, 10,000 official test images, and 100 fine classes.
CNN and Transformer	ResNet-18 is the CNN; Swin-T is the Transformer.
Scratch and pretrained	Both model families have a random-initialized `scratch` run and three ImageNet-pretrained runs.
Three freeze choices	`head`, `partial`, and `full` are implemented and measured for both families.
Metrics and cost	Table 4.1 gives validation accuracy, trainable parameters, time, and peak GPU memory.
Error analysis	A 20-superclass confusion matrix and real wrong-prediction images come from the selected model's one final test run.
Reproducible delivery	Code, JSON output, figures, this PDF, README, and a GitHub Pages A1 card are in the same repository.

1 CIFAR-100 Dataset and Protocol

1.1 Why this dataset is suitable

CIFAR-100 contains 60,000 colour images of size 32×32. It has 100 fine classes such as *apple*, *camel*, and *tulip*. Each fine class has 500 official training images and 100 official test images. The 100 fine classes are also grouped into 20 larger superclasses. For example, several animal classes belong to the same broad group. The large number of fine classes makes the task much harder than CIFAR-10 or Fashion-MNIST.

The original CIFAR page is the dataset source [1]. It describes the data and asks users to cite the technical report. The original public page does not state an explicit licence. Therefore, this report does not claim an MIT or commercial licence. The data and pretrained weights are used only for this educational coursework, with their source and limitations stated clearly. The local loader uses the public `uoft-cs/cifar100` Hugging Face copy for reliable automatic downloading; it keeps the original train/test partition.

1.2 Split, loader, and augmentation

The official 50,000-image training partition is split with seed 42. For every one of the 100 fine classes, the code shuffles its 500 indices, places 400 images in training and 100 in validation, and then joins the classes. This makes the split exactly balanced: 40,000 training images and 10,000 validation images. The official 10,000-image test partition is not mixed with either set.

The model expects 96×96 RGB input. A resize changes the tensor size but does not invent new visual information; it makes the input shape practical for the ImageNet-pretrained models. During training only, `RandomCrop` with padding and `RandomHorizontalFlip` create slightly changed versions of an image. Validation and test use only resize and normalization, so their measurements are repeatable. The ImageNet channel mean $(0.485, 0.456, 0.406)$ and standard deviation $(0.229, 0.224, 0.225)$ are used for all eight runs. This keeps the input rule the same for CNN and Transformer.

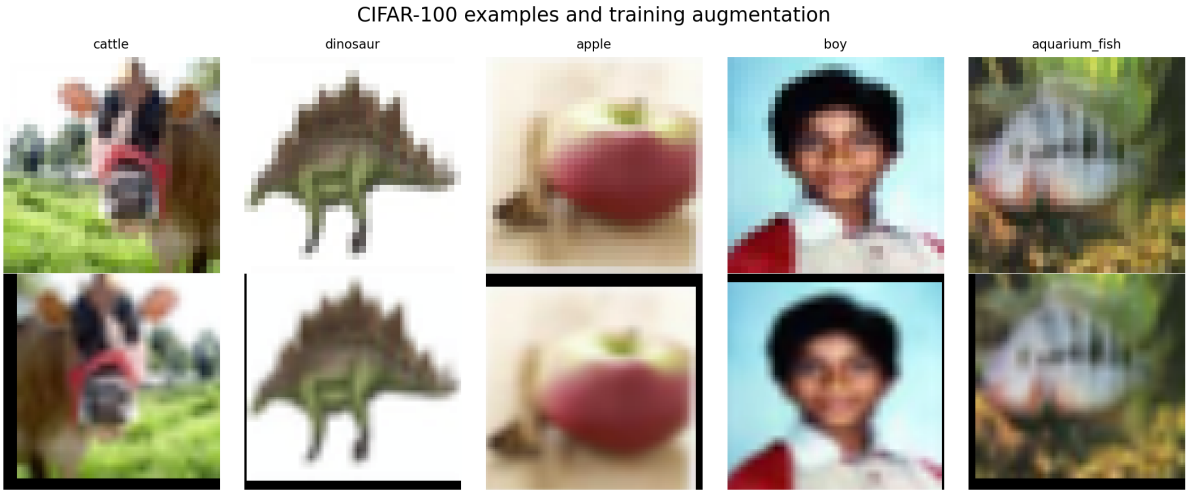


Figure 1.1: Five CIFAR-100 examples (top) and the random training augmentation applied to them (bottom). The random changes are used for training only.

1.3 Data representation and evaluation metrics

One input batch is $X \in \mathbb{R}^{B\times 3\times 96\times 96}$, where $B = 64$, 3 is the RGB channel count, and 96 is height and width after resize. The label y is an integer from 0 to 99. A model returns 100 logits $z = (z_1, \ldots, z_{100})$. Softmax changes logits into probabilities:

$$p_i = \frac{\exp(z_i)}{\sum_{j=1}^{100} \exp(z_j)}.$$

For true label y, cross-entropy loss is

$$\mathcal{L} = -\log(p_y).$$

In the code, `nn.CrossEntropyLoss()` receives logits directly and calculates this stable softmax-loss expression internally. A prediction is the class with the largest logit. Accuracy is

$$\text{accuracy} = \frac{\text{number of correct predictions}}{\text{number of evaluated images}}.$$

Macro-F1 calculates F1 separately for each of the 100 classes and then takes their unweighted mean. It gives every class equal importance, which is useful when a class-specific failure could be hidden by a single average accuracy.

2 ResNet-18 and Swin-T Theory

2.1 ResNet-18: the CNN model

ResNet-18 is a convolutional neural network with residual blocks [2]. A normal block learns a transformation $F(x, W)$ of its input x. A residual block adds the input back before the activation:

$$y = \mathrm{ReLU}(F(x, W) + x).$$

The shortcut $+x$ gives a direct path for information and gradients. In simple words, a layer can learn only the change it needs instead of rebuilding the whole representation. Convolutions read small nearby pixel regions, so ResNet naturally learns local edges, colours, and textures. After several stages, a final linear layer converts learned features into 100 class scores.

The torchvision ResNet-18 starts with an ImageNet classifier head containing 1,000 outputs. The code replaces that head with `nn.Linear(..., 100)`. This replacement is necessary because CIFAR-100 has 100 classes. In a frozen-backbone run, BatchNorm layers without trainable parameters are kept in evaluation mode. Their running mean and variance then remain frozen as well.

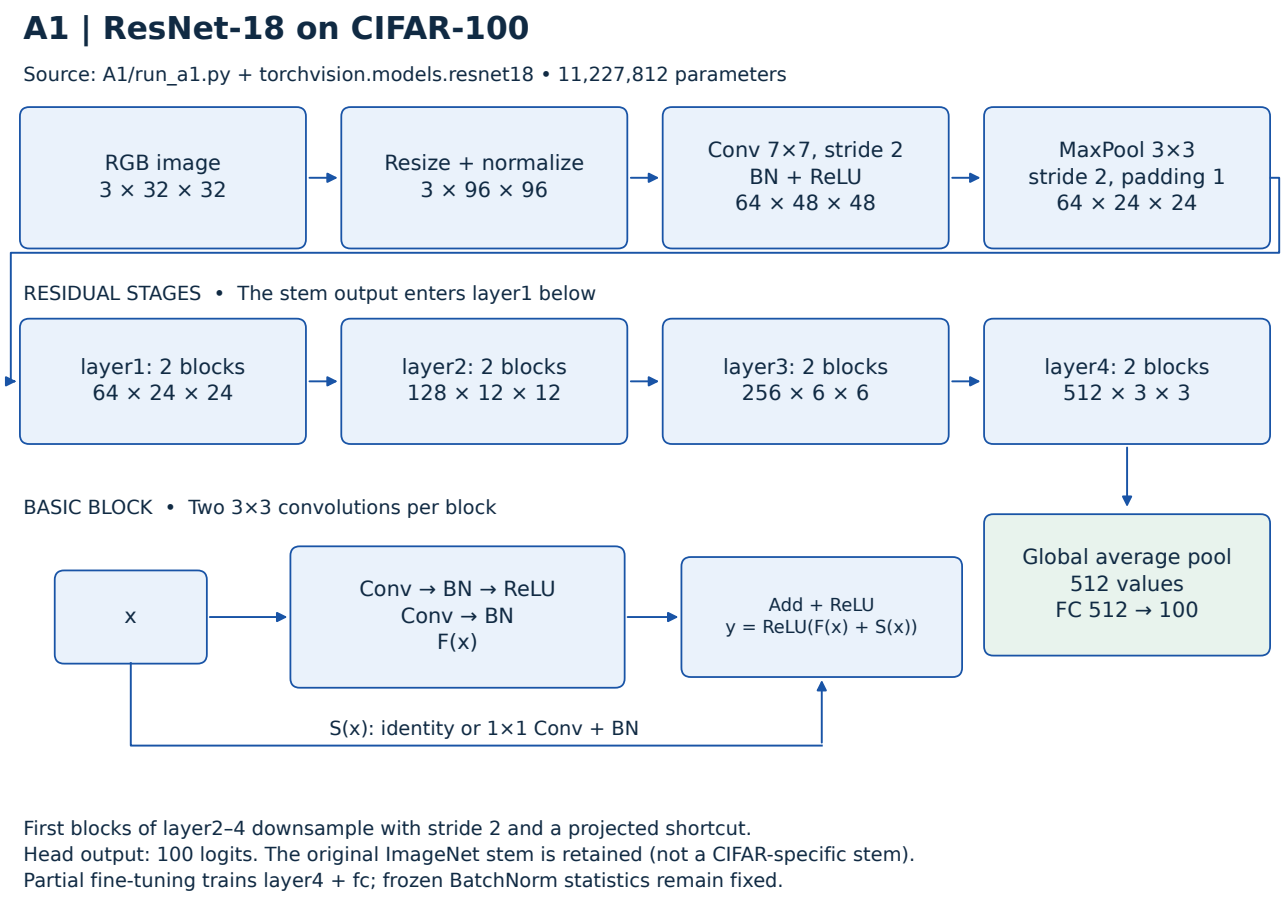


Figure 2.1: ResNet-18 as instantiated by `build_model` in `A1/run_a1.py`, following torchvision [6]. The original stem is retained, the input is resized to 96×96, and the final classifier has 100 outputs. Tensor shapes use $C \times H \times W$ and omit the batch dimension.

2.2 Swin-T: the Transformer model

Swin-T is a hierarchical vision Transformer based on shifted local windows [3]. First, the image is divided into non-overlapping patches. A patch is flattened and projected to a token vector:

$$t = W_p \,\mathrm{flatten}(x_{\mathrm{patch}}) + b_p.$$

The model forms query, key, and value matrices from tokens T:

$$Q = TW_Q, \qquad K = TW_K, \qquad V = TW_V.$$

Inside one local window, scaled dot-product attention is

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V.$$

The product QK^T compares one token with other tokens. Softmax converts comparisons to weights, and the weighted sum of V shares useful information. Using local windows avoids computing attention across every image patch at once. In the next block, shifted windows change the window boundaries, so information can pass between neighboring windows. This is the main Swin idea: it keeps attention efficient while building a larger visual context across stages.

Like ResNet, Swin-T has a final ImageNet head with 1,000 outputs. The code replaces `model.head` with a 100-output linear layer. Swin-T has more parameters than ResNet-18 and needs more memory/time, but it can use relations between patches through attention.

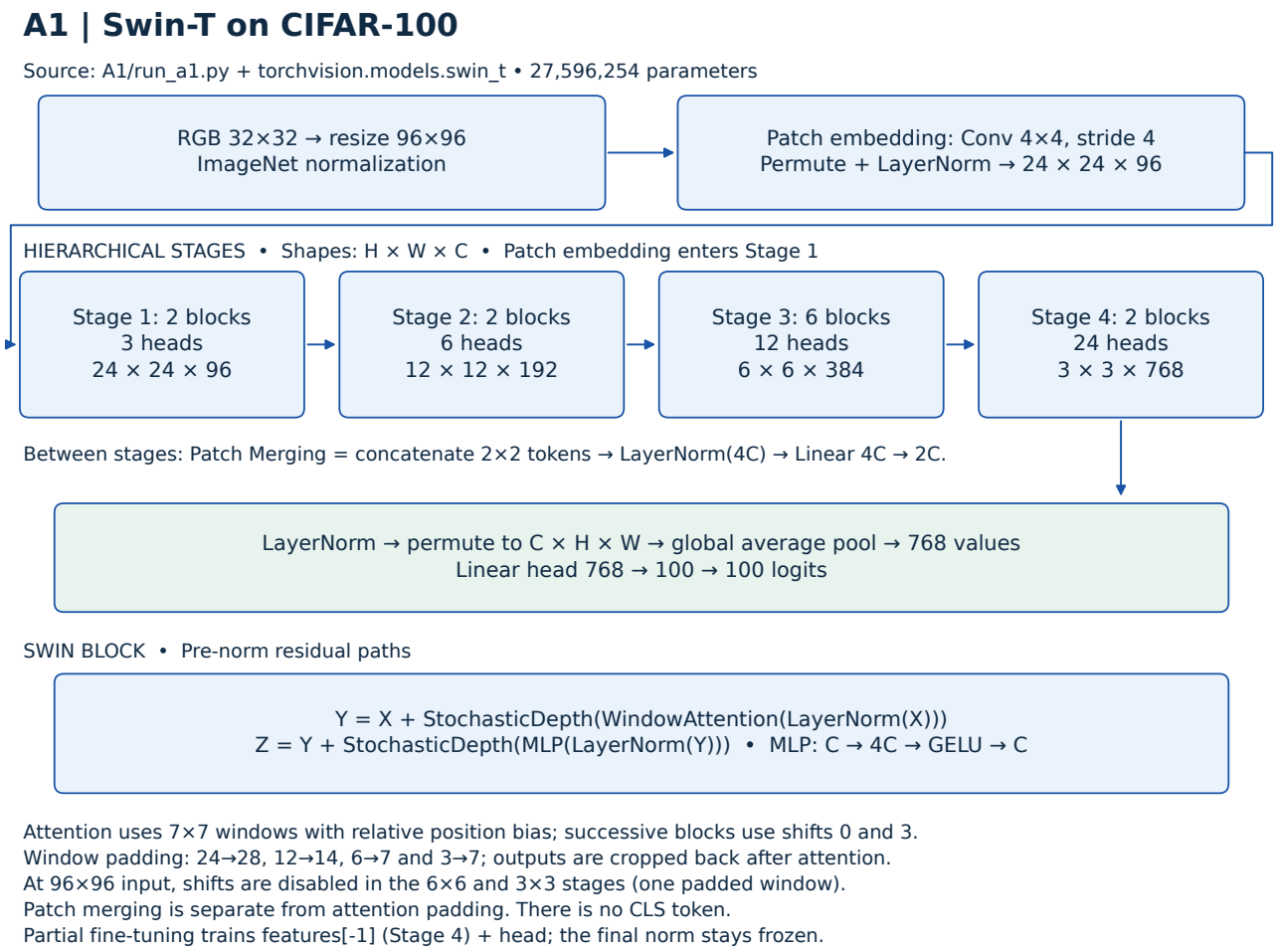


Figure 2.2: Swin-T as instantiated for A1, with stage depths $[2, 2, 6, 2]$, head counts $[3, 6, 12, 24]$, and a 100-class head. Stage shapes use $H \times W \times C$. This diagram follows torchvision's Swin-T implementation rather than a generic 224×224 ImageNet diagram.

At the project's 96×96 input size, the four stage grids are 24×24, 12×12, 6×6, and 3×3. Window attention pads these grids to multiples of seven and removes the padding afterward. In the last two stages the padded grid fits one window, so torchvision disables the spatial shift

there. Patch merging separately reduces the grid size between stages. The final LayerNorm and global average pooling produce 768 features before the new 100-class head. In `partial` mode, only `features[-1]` and `head` train; the final LayerNorm remains frozen. These details are visible in Figure 2.2 and follow the torchvision implementation [7].

2.3 Why pretrained weights matter

ImageNet pretraining gives parameters θ_0 that have already learned general visual patterns from a large image dataset. A random model instead starts at random θ_{random}. During fine-tuning, gradient-based optimization updates trainable parameters:

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L},$$

where η is the learning rate. AdamW implements a more stable version of this update with moving averages and weight decay, but the idea remains: reduce loss by moving parameters in the opposite direction of its gradient.

Pretraining does not solve the new task automatically because ImageNet has 1,000 different labels. The new 100-class head must learn CIFAR-100 labels. The experiment tests how much of the pretrained backbone should change while learning that head.

3 Transfer Learning and Freeze Modes

3.1 The four configurations

Each model family uses four configurations. The same data, split, batch size, and epoch count are used in every configuration. The mode changes only parameter initialization and which parameters receive gradients.

Table 3.1: A1 experiment matrix. `partial` means the last feature stage plus the new classifier head.

Family	Mode	**Initialization and trainable part**
`cnn`	`scratch`	ResNet-18 random initialization; all parameters train.
`cnn`	`head`	ImageNet ResNet-18; only `fc` classifier head trains.
`cnn`	`partial`	ImageNet ResNet-18; `layer4` and `fc` train.
`cnn`	`full`	ImageNet ResNet-18; every parameter trains.
`transformer`	`scratch`	Swin-T random initialization; all parameters train.
`transformer`	`head`	ImageNet Swin-T; only `head` classifier trains.
`transformer`	`partial`	ImageNet Swin-T; final `features` stage and `head` train.
`transformer`	`full`	ImageNet Swin-T; every parameter trains.

3.2 What freezing changes

For `head`, the backbone weights stay fixed. Only the new classifier learns. This is fast and uses little GPU memory because few gradients and optimizer states are stored. It works well when pretrained features already match the new images, but it cannot change the backbone if CIFAR-100 needs different features.

For `partial`, the early stages remain fixed and only the final stage is allowed to adapt. This is a middle choice: low-level edges and basic shapes are reused, while higher-level class features can change. It costs more than head-only training but less than full fine-tuning.

For `full`, every pretrained parameter receives gradients. It has the greatest capacity to adapt, but is also the most expensive. It may need a smaller learning rate so good pretrained features are not changed too abruptly. Here head-only uses learning rate 0.001, scratch and partial use 0.0003, and full uses 0.0001. These rates are written explicitly in the JSON result file.

3.3 Experiment Algorithm

The following steps correspond directly to `A1/run_a1.py`.

1. Set seed 42. Load CIFAR-100 train and test partitions, then make 400 training and 100 validation indices for every fine class.

2. Build a training loader with resize, random crop, random flip, tensor conversion, and normalization. Build validation and test loaders without random augmentation.

3. For one experiment row, call `build_model`. It loads ResNet-18 or Swin-T, optionally loads ImageNet weights, replaces the classifier by 100 outputs, and applies the selected freeze mode.

4. Count parameters with `requires_grad=True`. Start the timer and reset GPU peak-memory statistics.

5. For each of three epochs, read a batch, move images and labels to CUDA, call the model, and get 100 logits per image.

6. Compute `CrossEntropyLoss`. Clear gradients with `zero_grad()`, call `loss.backward()`, then call `optimizer.step()`.

7. Switch to evaluation mode and calculate validation loss and accuracy without gradients. Keep a CPU copy of the weights when validation accuracy improves.

8. Repeat Steps 3–7 for all eight rows. Do not inspect the official test partition while choosing a row.

9. Select the largest validation accuracy using `select_best_experiment`. Load its saved best-validation weights and evaluate the official test partition only after selection.

10. Save JSON metrics and figures: augmentation examples, learning curves, cost comparison, a 20-superclass confusion matrix, and wrong-prediction examples.

4 Measured Results and Discussion

4.1 Execution environment

All numbers in this section come from one executed script on an NVIDIA GeForce RTX 3050 Ti Laptop GPU. Inputs are 96×96, batch size is 64, and every row trains for three epochs. The output file `reports/a1_results.json` stores the unrounded values. Time measures the three training epochs and three validation passes for one configuration. Peak GPU memory is measured by PyTorch during that configuration.

Table 4.1: All eight measured validation experiments. The highest validation accuracy is bold.

Family	Mode	Validation accuracy	Trainable parameters	Time (s)	Peak memory (MB)
ResNet-18	scratch	0.3689	11,227,812	147.7	434.2
ResNet-18	head	0.5698	51,300	35.2	138.7
ResNet-18	partial	0.6930	8,445,028	48.2	235.2
ResNet-18	full	0.7441	11,227,812	91.3	434.2
Swin-T	scratch	0.1971	27,596,254	470.4	1,826.2
Swin-T	head	0.7076	76,900	169.1	330.4
Swin-T	partial	0.7573	14,260,756	234.8	493.4
Swin-T	full	**0.8281**	27,596,254	470.4	1,826.5

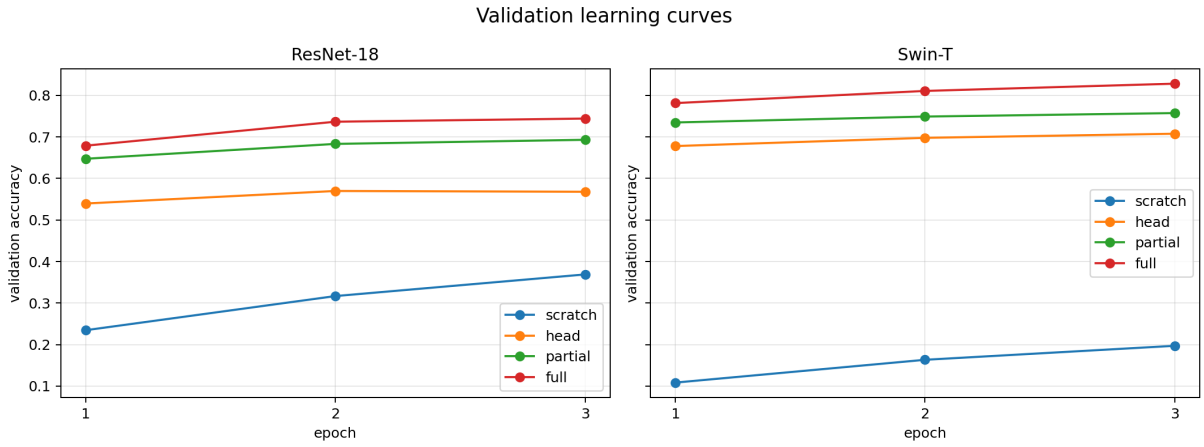


Figure 4.1: Validation accuracy after each epoch. Each panel compares the four freeze modes inside one model family.

The two scratch results are low: ResNet-18 reaches 0.3689 and Swin-T reaches 0.1971. This does not prove that a Transformer is always worse. It shows that a larger Transformer trained from random weights needs more optimization time and possibly a different schedule. In this controlled three-epoch setting, ResNet's local convolution prior learns CIFAR-100 faster from scratch.

Pretraining changes the result strongly. Head-only training improves ResNet-18 from 0.3689 to 0.5698 and Swin-T from 0.1971 to 0.7076. Swin-T head-only is slower than ResNet head-only because it still runs the complete Transformer forward pass, but it needs only 76,900 trainable parameters and 330.4 MB peak memory. This can be useful when GPU memory is limited.

4.2 Freeze comparison and training cost

For ResNet-18, full fine-tuning is best at 0.7441. Partial fine-tuning gives 0.6930 while training 8.45 million parameters instead of 11.23 million. Head-only is cheapest at 35.2 seconds but loses accuracy. This gives a simple trade-off: select head-only when a fast baseline is enough, partial when reducing cost matters, and full when the additional accuracy is worth the cost.

For Swin-T, the same order appears: head-only gives 0.7076, partial gives 0.7573, and full gives 0.8281. Full fine-tuning takes 470.4 seconds and about 1.83 GB peak memory. It is around five times the ResNet full-training time, so its higher accuracy has a real cost. The selected configuration is suitable for the available GPU, but it would be less practical on a CPU-only machine.

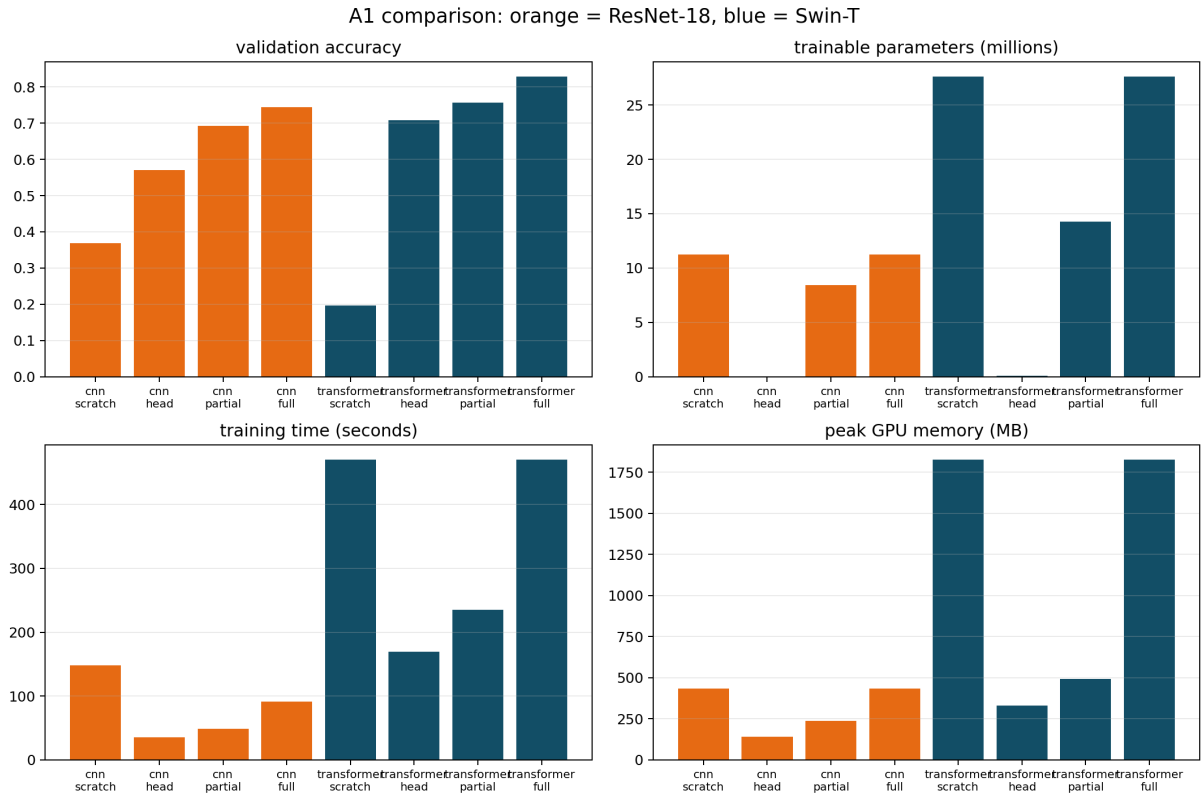


Figure 4.2: Validation accuracy, trainable parameters, training time, and peak GPU memory for all eight experiments. Orange bars are ResNet-18; blue bars are Swin-T.

4.3 Official test evaluation after validation selection

The largest validation accuracy is Swin-T full fine-tune at 0.8281. The saved checkpoint from the best validation epoch is evaluated on the untouched official test partition only after validation selects it. The recorded final test metrics are in Table 4.2. No other configuration receives a test metric for model selection, because comparing many test metrics and choosing the largest one would leak test information into selection.

Table 4.2: One official test evaluation after validation selection.

Selected model	**Test accuracy**	**Test macro-F1**
Swin-T, ImageNet pretrained, full fine-tune	**0.8173**	**0.8162**

After the recorded sweep, the already-selected Swin-T full configuration was run again only to regenerate the numbered confusion matrix in Figure 5.1. That post-selection reproduction gave test accuracy 0.8140, compared with 0.8173 in Table 4.2. The 0.0033 difference is reported openly: GPU training with parallel data loading can vary slightly between runs. This extra run did not choose or change the selected configuration.

5 Error Analysis, Limitations, and Reproduction

5.1 Confusion matrix and wrong predictions

The selected model predicts 100 fine classes. A 100×100 confusion matrix would be hard to read in a report, so Figure 5.1 groups true and predicted fine labels into CIFAR-100's 20 official superclasses. Each row is normalized, so the dark diagonal means that a broad group is usually predicted correctly. It is only a readability summary; the model itself still predicts the 100 fine labels. This figure uses the transparent post-selection reproduction described in Section 4.3; it is not used to select a model.

Figure 5.2 shows 12 real wrong predictions from the final test run. Several pairs have visual or semantic similarity: mountain versus road, cloud versus sea, tulip versus orchid, camel versus crocodile, and rabbit versus kangaroo. Some errors also show the basic problem of CIFAR-100: a 32×32 source image can contain little detail. Upsizing to 96×96 helps the network interface but cannot add missing pixels.

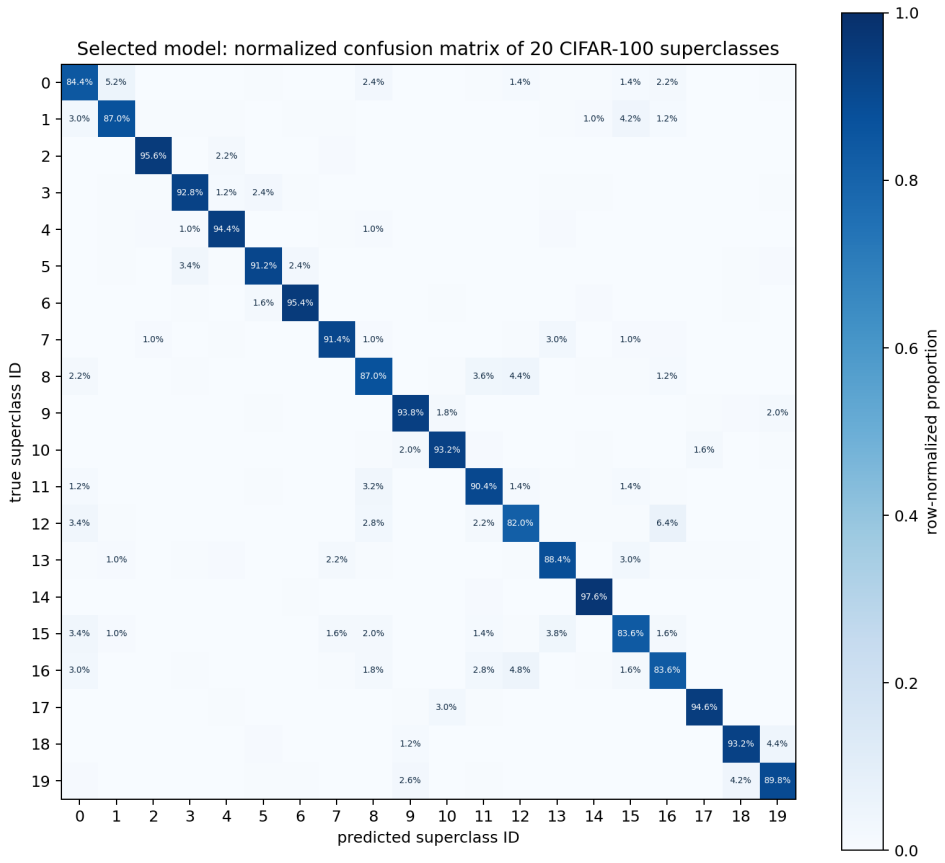


Figure 5.1: Row-normalized confusion matrix of the 20 CIFAR-100 superclasses for the selected model. Cells at or above 1% show their percentage; smaller cells are blank to keep the figure readable.

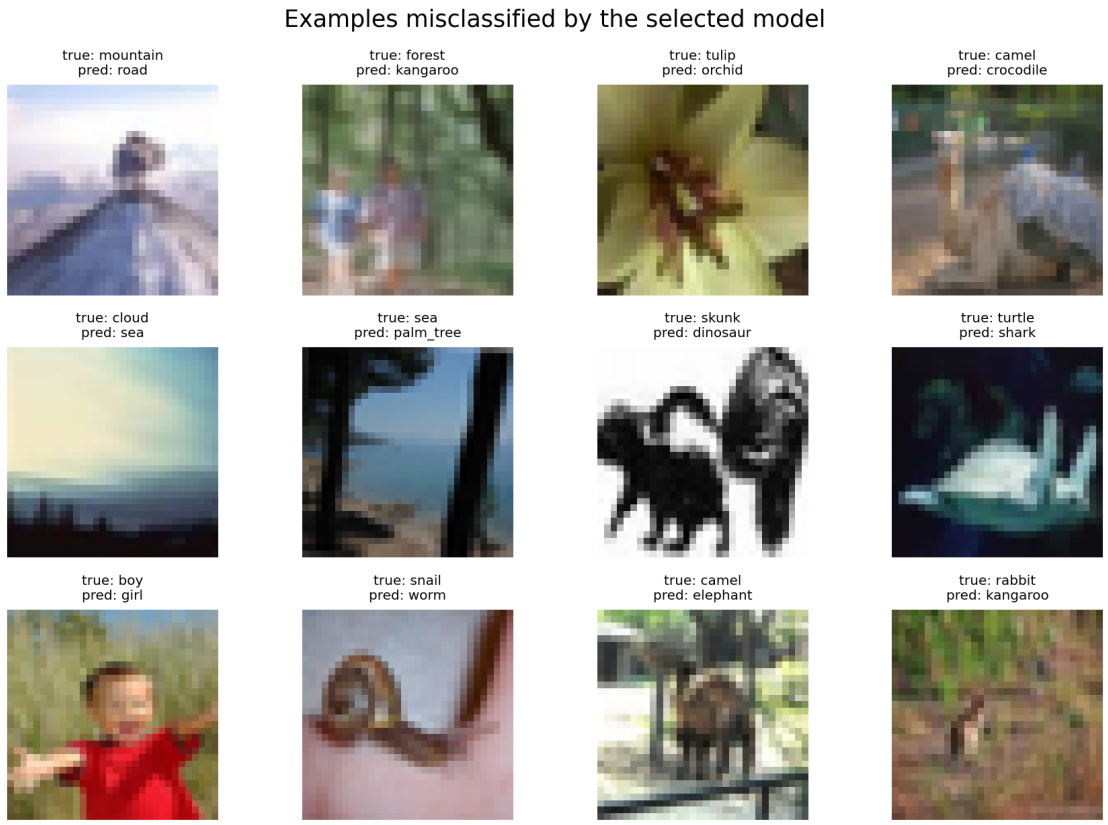


Figure 5.2: Twelve wrong fine-class predictions from the selected model's one final test evaluation.

5.2 Limitations

This is a clear A1 comparison, but it is not a maximum-performance CIFAR-100 study. First, every experiment has only three epochs. The low scratch results, especially Swin-T, would likely improve with a longer training schedule, a learning-rate schedule, and several random seeds. Second, CIFAR-100 source images are only 32×32; resizing to 96×96 is required for a practical pretrained-model interface but does not create new detail. Third, the results come from one GPU and one seed. Time and memory may change on another machine. Fourth, the source dataset page does not give an explicit licence, so this work should remain educational unless its terms are checked again for a different use.

5.3 How to reproduce

Install packages from the repository root and run:

```
pip install -r requirements.txt
python -m A1.run_a1
```

The first run downloads the public dataset and torchvision ImageNet weights. The script saves `reports/a1_results.json` and five PNG files under `reports/figures/`. The A1 README

explains the files in beginner-friendly order. The project page links directly to the code and this PDF.

6 Conclusion

This assignment compares two visual model families under the same CIFAR-100 split. With three epochs of random initialization, ResNet-18 learns faster than Swin-T. ImageNet pretraining improves both models substantially. For pretrained models, opening more layers increases both accuracy and cost. Swin-T full fine-tuning gives the best validation accuracy, 0.8281, and the selected model gives 0.8173 test accuracy with macro-F1 0.8162. ResNet-18 remains a smaller and faster alternative, while Swin-T gives the strongest measured result on the available GPU.

The code keeps the learning steps visible: make a balanced split, load and augment images, build a model, choose trainable parameters, calculate cross-entropy, call backward propagation, evaluate validation data, then test one selected model. This makes the project suitable for a new learner while still covering the complete A1 protocol.

References

[1] A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," University of Toronto, 2009. Dataset page: https://www.cs.toronto.edu/~kriz/cifar.html.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," *CVPR*, 2016. https://openaccess.thecvf.com/content_cvpr_2016/html/He_Deep_Residual_Learning_CVPR_2016_paper.html.

[3] Z. Liu et al., "Swin Transformer: Hierarchical Vision Transformer using Shifted Windows," 2021. https://arxiv.org/abs/2103.14030.

[4] PyTorch, "Torchvision models and pre-trained weights." https://docs.pytorch.org/vision/stable/models.html.

[5] Hugging Face, "uoft-cs/cifar100 dataset repository." https://huggingface.co/datasets/uoft-cs/cifar100.

[6] PyTorch, "Torchvision ResNet source code." https://docs.pytorch.org/vision/stable/_modules/torchvision/models/resnet.html.

[7] PyTorch, "Torchvision Swin Transformer source code." https://docs.pytorch.org/vision/stable/_modules/torchvision/models/swin_transformer.html.